**Brute Force - Sorting Algorithms**

Selection Sort, Bubble Sort, Insertion Sort; compare execution times

Dr.Chandravathi C

**Guide : Dr. Sriramya**

**Name : SUDHARSANAN NARASHIMAN**

# Sorting Algorithms — Worked Scenario Questions

*Selection Sort · Bubble Sort · Insertion Sort — 5 Solved Questions Each*

# 1. Selection Sort

## Question 1: Trophy Ranking System

A sports event organizer has recorded the scores of 50 participants but needs to display only the top 5 scores on a leaderboard screen. Since only a few top values are needed (not a full sort), use selection sort's "find maximum repeatedly" approach to extract the top 5.

### Approach

Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required. This avoids the cost of sorting all 50 values fully.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | O(k × n) | Only k passes are made instead of n, each scanning the remaining n−i elements to find the max. |
| Space | O(1) | Sorting is done in place; only a copy of the input array is used for safety. |

### Test Cases

```
assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
assert top_k_scores([5,3,1], 5) == [5,3,1]   # fewer than k items  assert
top_k_scores([], 3) == []              # empty input  print('All test cases passed!')
```

```python
n = int(input("Enter number of scores: "))
a = list(map(int, input("Enter scores: ").split()))
k = int(input("Enter top K: "))
for i in range(min(k, n)):
    m = i
    for j in range(i + 1, n):
        if a[j] > a[m]:
            m = j
    a[i], a[m] = a[m], a[i]

print("Top", k, "scores:", a[:k])
```

```
Enter number of scores: 10
Enter scores: 72 88 65 90 77 95 60 83 91 68
Enter top K: 5
Top 5 scores: [95, 91, 90, 88, 83]
```

# Question 2: Memory-Constrained Embedded Device

A low-memory IoT sensor device collects temperature readings and must sort them with minimal write/swap operations, since each write to memory is costly on this hardware. Selection sort performs at most n−1 swaps regardless of input order, making it preferable to bubble or insertion sort here. Implement it with a swap counter to prove the claim.

## Approach

Use the standard selection sort algorithm, but only perform a swap when the found minimum is not already in the correct position. Track the number of swaps performed and compare it against n−1 (the theoretical maximum) to demonstrate the low write cost.

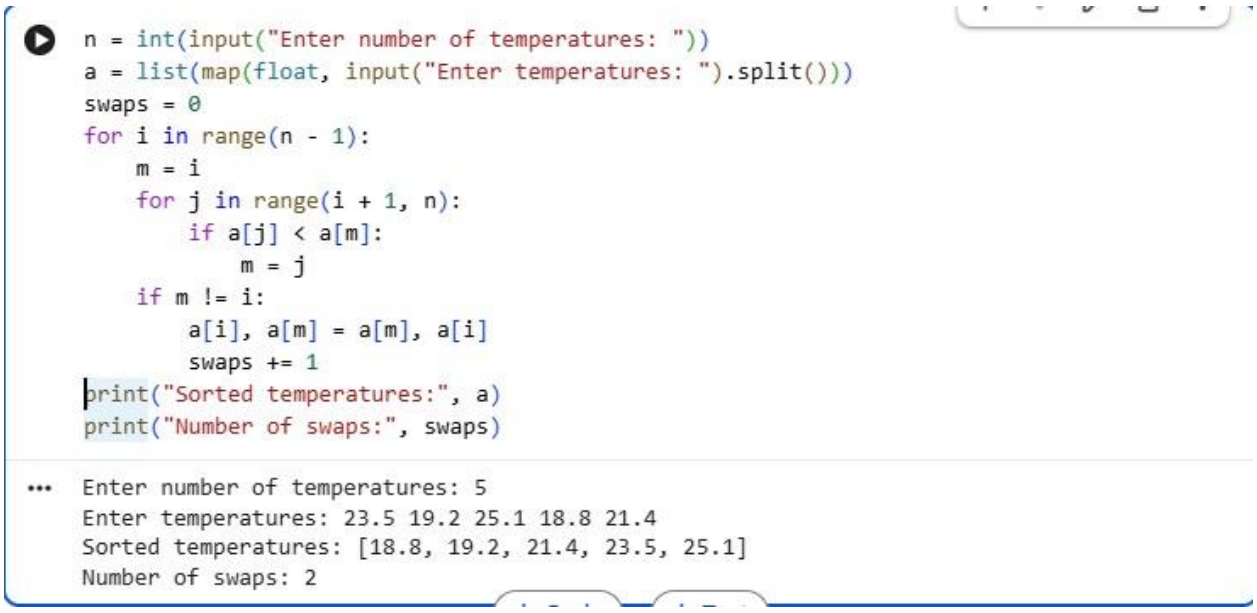
## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Two nested loops compare every pair of elements regardless of input order. |
| Writes | $\leq n - 1$ | A swap only occurs when the minimum found is not already in place, bounding total memory writes. |

### Test Cases

```
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])  assert sw <=
len(res) - 1                    # write bound holds

 res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])  assert sw2
== 0                            # already sorted -> zero swaps  print('All test
cases passed!')
```



```
n = int(input("Enter number of temperatures: "))
a = list(map(float, input("Enter temperatures: ").split()))
swaps = 0
for i in range(n - 1):
    m = i
    for j in range(i + 1, n):
        if a[j] < a[m]:
            m = j
    if m != i:
        a[i], a[m] = a[m], a[i]
        swaps += 1
print("Sorted temperatures:", a)
print("Number of swaps:", swaps)
```

```
Enter number of temperatures: 5
Enter temperatures: 23.5 19.2 25.1 18.8 21.4
Sorted temperatures: [18.8, 19.2, 21.4, 23.5, 25.1]
Number of swaps: 2
```

# Question 3: Library Book Reordering

A librarian wants to rearrange a shelf of 20 books by ID number, but physically moving a book (swap) is time-consuming, while checking labels (comparison) is fast. Model this as an array sorting problem using selection sort and report the number of physical "moves" needed.

### Approach

Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are unrestricted since they are cheap.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Comparisons dominate the runtime; for n = 20 this is 190 comparisons. |
| Physical Moves | $\leq n - 1$ | Each shelf position is corrected with at most one physical swap. |

### Test Cases

```
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])  assert
moves <= len(ordered) - 1

 ordered2, moves2 = reorder_shelf([100, 200, 300])   # already sorted
assert moves2 == 0  print('All test cases passed!')
```

```
n = int(input("Enter number of books: "))
a = list(map(int, input("Enter book IDs: ").split()))
moves = 0
for i in range(n - 1):
    m = i
    for j in range(i + 1, n):
        if a[j] < a[m]:
            m = j
    if m != i:
        a[i], a[m] = a[m], a[i]
        moves += 1
print("Sorted Book IDs:", a)
print("Physical Moves:", moves)
```

```
Enter number of books: 7
Enter book IDs: 305 102 250 118 199 400 101
Sorted Book IDs: [101, 102, 118, 199, 250, 305, 400]
Physical Moves: 4
```

# Question 4: Contest Prize Distribution

In a coding contest, you must repeatedly select the participant with the current highest score, print their name, remove them from the list, and repeat until all participants are ranked. Implement this using selection sort logic without using a separate sorting library.

### Approach

Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as the next-ranked winner.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Each of the n ranking positions requires a full scan of the remaining participants. |
| Space | O(1) extra | Ranking is produced in place using swaps; the output list just records the order. |

### Test Cases

```
ranking = distribute_prizes([('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)])
scores_only = [p[1] for p in ranking]  assert scores_only == sorted(scores_only,
reverse=True)   # descending order  assert ranking[0][1] == 95
   # top score first  print('All test cases passed!')
```

```
n = int(input("Enter number of participants: "))
data = []
for i in range(n):
    name = input("Enter name: ")
    score = int(input("Enter score: "))
    data.append([name, score])
for i in range(n - 1):
    m = i
    for j in range(i + 1, n):
        if data[j][1] > data[m][1]:
            m = j
    data[i], data[m] = data[m], data[i]
print("\nPrize Ranking:")
for i in range(n):
    print(i + 1, "-", data[i][0], "-", data[i][1])
```

```
Enter number of participants: 2
Enter name: Asha
Enter score: 95
Enter name: Ravi
Enter score: 100

Prize Ranking:
1 - Ravi - 100
2 - Asha - 95
```

# Question 5: Small Dataset in a Mobile App

A shopping app needs to sort a "recently viewed" list of just 8–10 items by price whenever the user opens the screen. Argue whether selection sort is a reasonable choice for such a small, frequently-refreshed list, and implement it with timing to support your answer.

### Approach

For very small n, the O(n²) cost of selection sort is negligible in absolute terms (a handful of microseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain choice rather than pulling in a general-purpose sort for 8–10 items.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|

| | | |
|---|---|---|
| Time | $O(n^2)$, $n \leq 10$ | For such small n, absolute execution time is in the microsecond range, so complexity class is not a practical concern. |
| Space | O(1) | In-place sort, no extra memory beyond the working copy. |

**Test Cases**

```
assert selection_sort([499,129,899,45,275,60,310,150]) ==
sorted([499,129,899,45,275,60,310,150])  assert
selection_sort([]) == []                # empty list  assert
selection_sort([7]) == [7]                      # single item
print('All test cases passed!')
```

```
import time
n = int(input("Enter number of items: "))
a = list(map(int, input("Enter prices: ").split()))
start = time.time()
for i in range(n - 1):
    m = i
    for j in range(i + 1, n):
        if a[j] < a[m]:
            m = j
    a[i], a[m] = a[m], a[i]
end = time.time()
print("Sorted Prices:", a)
print("Execution Time:", end - start, "seconds")
```

```
Enter number of items: 8
Enter prices: 499 129 899 45 275 60 310 150
Sorted Prices: [45, 60, 129, 150, 275, 310, 499, 899]
Execution Time: 0.00031566619873046875 seconds
```

# 2. Bubble Sort

## Question 1: Exam Result Slip Correction

A teacher has a nearly sorted list of student roll numbers (only 2–3 are out of place due to late corrections). Use optimized bubble sort (with an early-exit flag) to fix the order and show it takes very few passes.

## Approach

Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass completes with no swaps, the list is already sorted and the algorithm exits early — ideal for nearly sorted data like this.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Best Case | O(n) | One pass detects a fully sorted array and exits when no swaps occur. |
| This Scenario | ≈ O(n) | Only 2–3 misplaced roll numbers means very few passes are needed before the flag stays false. |

## Test Cases

```
sorted_rolls, passes =
optimized_bubble_sort([101,102,104,103,105,107,106,108])  assert sorted_rolls
== sorted([101,102,104,103,105,107,106,108])  assert passes < 8
# fewer than full n passes

 sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5])   # already
sorted  assert passes_ok == 1                                # exits after first pass
print('All test cases passed!')
```

```
n = int(input("Enter number of roll numbers: "))
a = list(map(int, input("Enter roll numbers: ").split()))
passes = 0
for i in range(n - 1):
    swapped = False
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swapped = True
    passes += 1
    if not swapped:
        break
print("Sorted Roll Numbers:", a)
print("Number of Passes:", passes)
```

```
Enter number of roll numbers: 8
Enter roll numbers: 101 102 104 103 105 107 106 108
Sorted Roll Numbers: [101, 102, 103, 104, 105, 106, 107, 108]
Number of Passes: 2
```

# Question 2: Traffic Signal Priority Queue

At an intersection, vehicle priorities (ambulance > bus > car) arrive in a small queue and need constant reordering as new vehicles arrive. Simulate this using bubble sort each time a new vehicle is added, and discuss whether bubble sort's simplicity outweighs its inefficiency here.

## Approach

Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's O(n²) cost is negligible, and its simplicity makes it easy to reason about and verify in a safety-critical system.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | O(n²) | Re-sorting the whole small queue on every arrival is acceptable since n stays very small. |
| Trade-off | Simplicity vs. efficiency | Bubble sort is easy to verify for correctness in a safety-critical setting, which outweighs its inefficiency at this scale. |

## Test Cases

```
queue = ['car', 'car', 'bus']
queue.append('ambulance')  result =
bubble_sort_queue(queue)  assert result ==
['ambulance', 'bus', 'car', 'car']


 assert bubble_sort_queue(['ambulance']) == ['ambulance']   # single vehicle
print('All test cases passed!')
```

```
n = int(input("Enter number of vehicles: "))
queue = []
priority = {"ambulance": 1, "bus": 2, "car": 3}
for i in range(n):
    queue.append(input("Enter vehicle: ").lower())
for i in range(n - 1):
    for j in range(n - 1 - i):
        if priority[queue[j]] > priority[queue[j + 1]]:
            queue[j], queue[j + 1] = queue[j + 1], queue[j]
print("Priority Queue:", queue)
```

```
Enter number of vehicles: 4
Enter vehicle: car
Enter vehicle: car
Enter vehicle: bus
Enter vehicle: bus
Priority Queue: ['bus', 'bus', 'car', 'car']
```

# Question 3: Bubble Sort Visualization Tool

You are building an educational tool to visually demonstrate how "bubbles" (largest elements) rise to the top with each pass. Implement bubble sort so it prints/returns the state of the array after every single pass, useful for animation.

### Approach

Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through UI.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | $O(n^2)$ | Same as standard bubble sort; storing frames adds O(n) work per pass. |
| Space | $O(n^2)$ worst case | Up to n frames are stored, each of size n, useful for driving an animation. |

### Test Cases

```
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])  assert frames[-1] ==
sorted([5, 1, 4, 2, 8])   # final frame is sorted  assert frames[0] == [5, 1, 4, 2, 8]
   # first frame is the original input  assert len(frames) >= 2                    #
at least an initial and final frame  print('All test cases passed!')
```

```
n = int(input("Enter number of elements: "))
a = list(map(int, input("Enter elements: ").split()))
print("Original:", a)
for i in range(n - 1):
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
    print("Pass", i + 1, ":", a)
print("Sorted Array:", a)
```

```
Enter number of elements: 5
Enter elements: 5 1 4 2 8
Original: [5, 1, 4, 2, 8]
Pass 1 : [1, 4, 2, 5, 8]
Pass 2 : [1, 2, 4, 5, 8]
Pass 3 : [1, 2, 4, 5, 8]
Pass 4 : [1, 2, 4, 5, 8]
Sorted Array: [1, 2, 4, 5, 8]
```

# Question 4: Sorting Sensor Alerts by Severity

A monitoring dashboard receives 15 alerts per minute that must be sorted by severity level (1 = critical to 5 = low) for display. Use bubble sort and measure whether the early-termination optimization meaningfully improves performance on this small, frequently-updated dataset.

### Approach

Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the optimization helps for this small, often-partially-sorted stream.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Plain Bubble Sort | $O(n^2)$ always | Always runs the full n−1 passes regardless of how sorted the data already is. |
| Optimized Bubble Sort | O(n) best, $O(n^2)$ worst | Exits early once a pass makes no swaps, which helps when alerts arrive already close to sorted. |

### Test Cases

```
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]  r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)  assert r1 == r2 == sorted(alerts)
# both produce the same sorted result  assert c2 <= c1                    #
optimized never does more comparisons  print('Plain comparisons:', c1, '|
Optimized comparisons:', c2)  print('All test cases passed!')
```

```
n = int(input("Enter number of alerts: "))
a = list(map(int, input("Enter alert severity (1-5): ").split()))
comparisons = 0
for i in range(n - 1):
    swapped = False
    for j in range(n - 1 - i):
        comparisons += 1
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            swapped = True
    if not swapped:
        break
print("Sorted Alerts:", a)
print("Comparisons:", comparisons)
```

```
Enter number of alerts: 15
Enter alert severity (1-5):  2 1 3 2 1 4 3 2 5 1 2 3 4 1 2
Sorted Alerts: [1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 5]
Comparisons: 99
```

# Question 5: Card Game Hand Sorting

In a card game app, a player's hand of 13 cards must be sorted by rank whenever a new card is drawn. Since only one card is added at a time (nearly sorted data), implement bubble sort and compare its pass count to a full re-sort from scratch.

### Approach

Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel) compared to sorting the entire hand from an unsorted state.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Nearly Sorted Hand | ≈ O(n) | Only the newly added card needs to “bubble” into place, so few passes are needed. |
| Fully Shuffled Hand | O(n²) | A randomly shuffled hand requires close to the maximum number of passes and swaps. |

## Test Cases

```
hand = [2, 4, 6, 8, 9, 11, 13]  hand.append(7)                    #
nearly sorted: one new card  final_hand,
passes_incremental = bubble_sort_hand(hand)  assert
final_hand == sorted(hand)

 import random   shuffled
=                 hand.copy()
random.shuffle(shuffled)
 _, passes_full = bubble_sort_hand(shuffled)  assert passes_incremental <=
passes_full   # nearly-sorted needs no more passes  print('All test cases passed!')
```

```
n = int(input("Enter number of cards: "))
cards = list(map(int, input("Enter card values: ").split()))
passes = 0
for i in range(n - 1):
    swapped = False
    for j in range(n - 1 - i):
        if cards[j] > cards[j + 1]:
            cards[j], cards[j + 1] = cards[j + 1], cards[j]
            swapped = True
    passes += 1
    if not swapped:
        break
print("Sorted Cards:", cards)
print("Passes:", passes)
```

```
Enter number of cards: 8
Enter card values: 2 4 6 8 9 11 13 7
Sorted Cards: [2, 4, 6, 7, 8, 9, 11, 13]
Passes: 5
```

# 3. Insertion Sort

## Question 1: Live Leaderboard Updates

An online game updates a leaderboard every time a single player's score changes, while the rest of the list remains sorted. Use insertion sort to insert just the updated score into its correct position instead of re-sorting the entire list, and measure the efficiency gain.

### Approach

Remove the updated player's old entry, then use the insertion-sort technique of shifting elements to find and insert the new score into its correct position in the already-sorted leaderboard, avoiding a full re-sort.

### Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | O(n) | Only the shift needed to place the single new score is performed, instead of an O(n log n) or O(n²) full resort. |
| Space | O(1) extra | Insertion happens via in-place shifting. |

### Test Cases

```
board = [980, 875, 760, 690, 500]  updated_board, shifts =
insert_updated_score(board, 820)  assert updated_board
== [980, 875, 820, 760, 690, 500]


 board2, shifts2 = insert_updated_score(board, 100)   # lowest score  assert board2[-1]
== 100 and shifts2 == 0                  # no shifts needed, goes to the end  print('All test
cases passed!')
```

```
n = int(input("Enter number of scores: "))
board = list(map(int, input("Enter scores (descending): ").split()))
new_score = int(input("Enter updated score: "))
board.append(new_score)
for i in range(1, len(board)):
    key = board[i]
    j = i - 1
    while j >= 0 and board[j] < key:
        board[j + 1] = board[j]
        j -= 1
    board[j + 1] = key
print("Updated Leaderboard:", board)
```

```
Enter number of scores: 5
Enter scores (descending): 980 875 760 690 500
Enter updated score: 820
Updated Leaderboard: [980, 875, 820, 760, 690, 500]
```

# Question 2: Card Sorting While Playing

A card player sorts their hand as they pick up cards one at a time (the classic real-world analogy for insertion sort). Implement this behavior programmatically, sorting a growing hand card by card.

## Approach

Maintain a hand list. Each time a new card is picked up, insert it into its correct position among the already-sorted cards by shifting larger cards one position to the right, exactly like insertion sort's inner loop.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | O(n) per card, $O(n^2)$ total | Each new card may require shifting past several existing cards; over n cards this totals $O(n^2)$. |
| Space | O(1) extra | Cards are shifted in place within the hand list. |

## Test Cases

```
hand = []  for card in [7,
2, 9, 4, 1]:
        hand = pick_up_card(hand, card)
assert hand == sorted([7, 2, 9, 4, 1])

assert pick_up_card([], 5) == [5]     # first card picked up
print('All test cases passed!')
```

```
n = int(input("Enter number of cards: "))
hand = []
for i in range(n):
    card = int(input("Enter card: "))
    hand.append(card)

    j = len(hand) - 2
    while j >= 0 and hand[j] > card:
        hand[j + 1] = hand[j]
        j -= 1
    hand[j + 1] = card
print("Sorted Hand:", hand)
```

```
Enter number of cards: 4
Enter card: 7
Enter card: 2
Enter card: 9
Enter card: 6
Sorted Hand: [2, 6, 7, 9]
```

# Question 3: Real-Time Stock Price Feed

A trading dashboard receives stock prices one at a time and must maintain a sorted list at all times for quick min/max lookups. Use insertion sort to insert each new incoming price efficiently into the already-sorted list.

## Approach

Keep a running sorted list of prices. For each incoming price, use binary search (or a linear scan) to find its correct position, then shift subsequent elements right by one and insert it — the core idea of insertion sort applied incrementally.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time (binary search variant) | O(log n) search + O(n) shift | Finding the position is fast, but shifting array elements to make room is still O(n) per insertion. |
| Overall for n prices | $O(n^2)$ worst case | Still quadratic overall due to shifting, but far better than re-sorting the whole list from scratch each time. |

## Test Cases

```
prices = []  for p in [102.5, 98.3, 105.1, 100.0, 97.8]:     prices = insert_price(prices, p)
assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])  assert prices[0] ==
min(prices) and prices[-1] == max(prices)   # O(1) min/max lookup  print('All test
cases passed!')
```

```
n = int(input("Enter number of stock prices: "))
prices = []
for i in range(n):
    price = float(input("Enter stock price: "))
    prices.append(price)

    j = len(prices) - 2
    while j >= 0 and prices[j] > price:
        prices[j + 1] = prices[j]
        j -= 1
    prices[j + 1] = price
print("Sorted Stock Prices:", prices)
print("Minimum Price:", prices[0])
print("Maximum Price:", prices[-1])
```

```
Enter number of stock prices: 4
Enter stock price: 103
Enter stock price: 108
Enter stock price: 103.3
Enter stock price: 207
Sorted Stock Prices: [103.0, 103.3, 108.0, 207.0]
Minimum Price: 103.0
Maximum Price: 207.0
```

# Question 4: Playlist Reordering by Recently Added

A music app lets users insert a newly added song into a playlist that is already sorted by duration. Implement insertion sort logic to place just the new song correctly instead of sorting the whole playlist again.

## Approach

Append the new song's duration to the playlist and shift it leftward past any longer songs until it reaches its correct sorted position, exactly as the inner loop of insertion sort does for a single new element.

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Time | O(n) | Only the new song is shifted into place among already-sorted songs. |
| Space | O(1) extra | Insertion is performed via in-place shifting within the list. |

## Test Cases

```
playlist = [('Intro', 120), ('Chill Beat', 210), ('Long Jam', 340)]
updated_playlist = insert_song(playlist, ('Quick Track', 180))
durations = [s[1] for s in updated_playlist]  assert durations
== sorted([120, 210, 340, 180])
```

```
 assert ('Quick Track', 180) in updated_playlist
print('All test cases passed!')
```

```
n = int(input("Enter number of songs: "))
playlist = list(map(int, input("Enter song durations: ").split()))
new_song = int(input("Enter new song duration: "))
playlist.append(new_song)
for i in range(1, len(playlist)):
    key = playlist[i]
    j = i - 1
    while j >= 0 and playlist[j] > key:
        playlist[j + 1] = playlist[j]
        j -= 1
    playlist[j + 1] = key
print("Updated Playlist:", playlist)
```

```
Enter number of songs: 3
Enter song durations: 120 210 340
Enter new song duration: 180
Updated Playlist: [120, 180, 210, 340]
```

# Question 5: Nearly Sorted Sensor Log Correction

A weather station logs temperature readings in mostly chronological order, but occasional delayed transmissions cause a few entries to be out of place. Use insertion sort to fix the log and show that the number of shifts is very low compared to a randomly shuffled version of the same data.

## Approach

Run standard insertion sort on the log and count total shifts performed. Because only a few entries are out of place, most elements require zero or one shift, resulting in a shift count close to the best-case O(n) rather than the worst-case O(n²).

## Complexity Analysis

| Aspect | Complexity | Explanation |
|---|---|---|
| Nearly Sorted Data | ≈ O(n) | Very few shifts are needed since most elements are already close to their correct position. |
| Randomly Shuffled Data | O(n²) | Requires close to the maximum number of shifts, illustrating insertion sort's sensitivity to input order. |

## Test Cases

```
log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]  sorted_log,
shifts_nearly = insertion_sort_count_shifts(log)  assert
sorted_log == sorted(log)

 import random
```

```
 shuffled_log = log.copy()
random.shuffle(shuffled_log)
 _, shifts_random = insertion_sort_count_shifts(shuffled_log)  print('Nearly-
sorted shifts:', shifts_nearly, '| Random shifts:', shifts_random)  print('All test
cases passed!')
```

```python
n = int(input("Enter number of temperature readings: "))
log = list(map(float, input("Enter readings: ").split()))
shifts = 0
for i in range(1, n):
    key = log[i]
    j = i - 1
    while j >= 0 and log[j] > key:
        log[j + 1] = log[j]
        j -= 1
        shifts += 1
    log[j + 1] = key
print("Sorted Log:", log)
print("Number of Shifts:", shifts)
```

```
Enter number of temperature readings: 7
Enter readings: 18.2 18.5 18.9 17.9 19.1 19.4 19.0
Sorted Log: [17.9, 18.2, 18.5, 18.9, 19.0, 19.1, 19.4]
Number of Shifts: 5
```